

EXPT 1 : Caesar Cipher

```
text = "HELLO"
```

```
shift = 3
```

```
cipher = ""
```

```
for ch in text:
```

```
    cipher += chr((ord(ch) - 65 + shift) % 26 + 65)
```

```
print("Cipher Text:", cipher)
```

```
>> type help , copyright ,
=====
>> Cipher Text: KHOOR
>> |
```

EXPT 2 : Monoalphabetic Substitution Cipher

```
# Monoalphabetic Substitution Cipher
```

```
plain_alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
cipher_alphabet = "QWERTYUIOPASDFGHJKLZXCVBNM"
```

```
plaintext = input("Enter Plain Text: ").upper()
```

```
ciphertext = ""
```

```
for ch in plaintext:
```

```

if ch in plain_alphabet:
    index = plain_alphabet.index(ch)
    ciphertext += cipher_alphabet[index]
else:
    ciphertext += ch

print("Cipher Text:", ciphertext)

```

```

=====
Enter Plain Text: GOOD MORNING
Cipher Text: UGGR DGKFOFU
|

```

EXPT 3 : Playfair Cipher

```

def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []

    for ch in key:
        if ch not in matrix and ch.isalpha():
            matrix.append(ch)

    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
        if ch not in matrix:
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]

```

```

def find_position(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

def encrypt(text, matrix):
    text = text.upper().replace("J", "I").replace(" ", "")

    if len(text) % 2 != 0:
        text += "X"

    cipher = ""

    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]

        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)

        if r1 == r2:
            cipher += matrix[r1][(c1+1)%5]
            cipher += matrix[r2][(c2+1)%5]
        elif c1 == c2:
            cipher += matrix[(r1+1)%5][c1]
            cipher += matrix[(r2+1)%5][c2]
        else:
            cipher += matrix[r1][c2]

```

```

        cipher += matrix[r2][c1]

    return cipher

key = input("Enter Key: ")
text = input("Enter Plain Text: ")

matrix = generate_matrix(key)

print("Cipher Text:", encrypt(text, matrix))

```

```

=====
Enter Key: 7
Enter Plain Text: HELLO
Cipher Text: KCMMNY
>

```

EXPT 4: Polyalphabetic Substitution Cipher

Polyalphabetic Substitution Cipher

```

plaintext = input("Enter Plain Text: ").upper()
key = input("Enter Key: ").upper()

ciphertext = ""

for i in range(len(plaintext)):
    p = ord(plaintext[i]) - ord('A')

```

```
k = ord(key[i % len(key)]) - ord('A')
```

```
c = (p + k) % 26
```

```
ciphertext += chr(c + ord('A'))
```

```
print("Cipher Text:", ciphertext)
```

```
=====
Enter Plain Text: HI HELLO
Enter Key: 5
Cipher Text: VWHVSZC
>>>
```

EXPT 5: ENCRYPTION - Affine Cipher

Affine Cipher Encryption

```
a = 5
```

```
b = 8
```

```
plaintext = input("Enter Plain Text: ").upper()
```

```
ciphertext = ""
```

```
for ch in plaintext:
```

```
    if ch.isalpha():
```

```
        x = ord(ch) - ord('A')
```

```
        c = (a * x + b) % 26
```

```
        ciphertext += chr(c + ord('A'))
```

```
    else:
```

```
        ciphertext += ch
```

```
print("Cipher Text:", ciphertext)
```

```
=====
Enter Plain Text: GOOD AFTERNOON
Cipher Text: MAAH IHZCPVAAV
>
```

EXPT 5 :DECRYPTION - Affine Cipher

```
# Affine Cipher Decryption
```

```
a = 5
```

```
b = 8
```

```
a_inv = 21 # Multiplicative inverse of 5 mod 26
```

```
ciphertext = input("Enter Cipher Text: ").upper()
```

```
plaintext = ""
```

```
for ch in ciphertext:
```

```
    if ch.isalpha():
```

```
        y = ord(ch) - ord('A')
```

```
        p = (a_inv * (y - b)) % 26
```

```
        plaintext += chr(p + ord('A'))
```

```
    else:
```

```
        plaintext += ch
```

```
print("Plain Text:", plaintext)
```

```
Enter Cipher Text: MAAX IHZCPVAAV
Plain Text: GOOD AFTERNOON
> |
```

EXPT 6: Affine Cipher Cryptanalysis

Affine Cipher Cryptanalysis

```
cipher = input("Enter Cipher Text: ").upper()
```

```
for a in [1,3,5,7,9,11,15,17,19,21,23,25]:
```

```
    # Find inverse of a
```

```
    for i in range(26):
```

```
        if (a * i) % 26 == 1:
```

```
            a_inv = i
```

```
            break
```

```
for b in range(26):
```

```
    plain = ""
```

```
    for ch in cipher:
```

```
        if ch.isalpha():
```

```
            y = ord(ch) - 65
```

```
            x = (a_inv * (y - b)) % 26
```

```
            plain += chr(x + 65)
```

```
print("a =", a, "b =", b, "-->", plain)
```

Enter Cipher Text: RCLLA

a = 1 b = 0 --> RCLLA
a = 1 b = 1 --> QBKKZ
a = 1 b = 2 --> PAJJY
a = 1 b = 3 --> OZIIX
a = 1 b = 4 --> NYHHW
a = 1 b = 5 --> MXGGV
a = 1 b = 6 --> LWFFU
a = 1 b = 7 --> KVEET
a = 1 b = 8 --> JUDDS
a = 1 b = 9 --> ITCCR
a = 1 b = 10 --> HSBBQ
a = 1 b = 11 --> GRAAP
a = 1 b = 12 --> FQZZO
a = 1 b = 13 --> EPYYN
a = 1 b = 14 --> DOXXM
a = 1 b = 15 --> CNWWL
a = 1 b = 16 --> BMVVK
a = 1 b = 17 --> ALUUJ
a = 1 b = 18 --> ZKTTI
a = 1 b = 19 --> YJSSH
a = 1 b = 20 --> XIRRG
a = 1 b = 21 --> WHQQF
a = 1 b = 22 --> VGPPE
a = 1 b = 23 --> UFOOD
a = 1 b = 24 --> TENNC
a = 1 b = 25 --> SDMMB
a = 3 b = 0 --> XSVVA
a = 3 b = 1 --> OJMMR
a = 3 b = 2 --> FADDI
a = 3 b = 3 --> WRUUZ
a = 3 b = 4 --> NILLQ
a = 3 b = 5 --> EZCCH
a = 3 b = 6 --> VQTTY

EXPT 7: Simple Substitution Cipher Decryption

Simple Substitution Cipher Decryption

plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"


```
ciphertext = input("Enter Cipher Text: ").upper()
```

```
plaintext = ""
```

```
for ch in ciphertext:
```

```
    if ch in cipher:
```

```
        index = cipher.index(ch)
```

```
        plaintext += plain[index]
```

```
    else:
```

```
        plaintext += ch
```

```
print("Plain Text:", plaintext)
```

```
=====
Enter Cipher Text: ITSSG
Plain Text: HELLO
|
```

EXPT 8: ENCRYPTION- Keyword-Based Substitution Cipher Encryption

```
# Keyword-Based Substitution Cipher Encryption
```

```
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
cipher = "SECR TABDFGHIJKLMNOPQUVWXYZ"
```

```
text = input("Enter Plain Text: ").upper()
```

```
result = ""
```

```
for ch in text:
```

```
    if ch in plain:
```

```
        index = plain.index(ch)
```

```

        result += cipher[index]
    else:
        result += ch

print("Cipher Text:", result)

```

```

Enter Plain Text: HELLO
Cipher Text: DTIIL
>

```

EXPT 8 : DECRYPTION- Keyword-Based Substitution Cipher Decryption

Keyword-Based Substitution Cipher Decryption

```

plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "SECRTABDFGHIJKLMNOPQUVWXYZ"

```

```

text = input("Enter Cipher Text: ").upper()

```

```

result = ""

```

```

for ch in text:
    if ch in cipher:
        index = cipher.index(ch)
        result += plain[index]
    else:
        result += ch

```

```

print("Plain Text:", result)

```

```
=====
Enter Cipher Text: DTIIL
Plain Text: HELLO
|
```

EXPT 9: Playfair Cipher Decryption

Playfair Cipher Decryption

```
def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []

    for ch in key:
        if ch not in matrix and ch.isalpha():
            matrix.append(ch)

    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
        if ch not in matrix:
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]

def find_pos(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

def decrypt(cipher, matrix):
```

```
plain = ""
```

```
for i in range(0, len(cipher), 2):
```

```
    a = cipher[i]
```

```
    b = cipher[i+1]
```

```
    r1, c1 = find_pos(matrix, a)
```

```
    r2, c2 = find_pos(matrix, b)
```

```
    if r1 == r2: # Same row
```

```
        plain += matrix[r1][(c1-1) % 5]
```

```
        plain += matrix[r2][(c2-1) % 5]
```

```
    elif c1 == c2: # Same column
```

```
        plain += matrix[(r1-1) % 5][c1]
```

```
        plain += matrix[(r2-1) % 5][c2]
```

```
    else: # Rectangle
```

```
        plain += matrix[r1][c2]
```

```
        plain += matrix[r2][c1]
```

```
    return plain
```

```
key = input("Enter Key: ")
```

```
cipher = input("Enter Cipher Text: ").upper()
```

```
matrix = generate_matrix(key)
```

```
print("Plain Text:", decrypt(cipher, matrix))
```

```
=====
Enter Key: MONARCHY
Enter Cipher Text: CFSUPM
Plain Text: HELXLO
.
```

EXPT 10 : Playfair Cipher Encryption

Playfair Cipher Encryption

```
def generate_matrix(key):
```

```
    key = key.upper().replace("J", "I")
```

```
    matrix = []
```

```
    for ch in key:
```

```
        if ch not in matrix and ch.isalpha():
```

```
            matrix.append(ch)
```

```
    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
```

```
        if ch not in matrix:
```

```
            matrix.append(ch)
```

```
    return [matrix[i:i+5] for i in range(0, 25, 5)]
```

```
def find_pos(matrix, ch):
```

```
    for i in range(5):
```

```
        for j in range(5):
```

```
            if matrix[i][j] == ch:
```

```
                return i, j
```

```

def prepare_text(text):

    text = text.upper().replace("J", "I").replace(" ", "")

    result = ""

    i = 0

    while i < len(text):

        a = text[i]

        if i + 1 < len(text):

            b = text[i + 1]

            if a == b:

                result += a + "X"

                i += 1

            else:

                result += a + b

                i += 2

        else:

            result += a + "X"

            i += 1

    return result

```

```

def encrypt(text, matrix):

    cipher = ""

    for i in range(0, len(text), 2):

        a, b = text[i], text[i+1]

        r1, c1 = find_pos(matrix, a)

```

```

r2, c2 = find_pos(matrix, b)

if r1 == r2:    # Same row
    cipher += matrix[r1][(c1 + 1) % 5]
    cipher += matrix[r2][(c2 + 1) % 5]

elif c1 == c2: # Same column
    cipher += matrix[(r1 + 1) % 5][c1]
    cipher += matrix[(r2 + 1) % 5][c2]

else:          # Rectangle rule
    cipher += matrix[r1][c2]
    cipher += matrix[r2][c1]

return cipher

key = input("Enter Key: ")
plaintext = input("Enter Plain Text: ")

matrix = generate_matrix(key)
plaintext = prepare_text(plaintext)

print("Cipher Text:", encrypt(plaintext, matrix))

```

```

=====
Enter Key: MONARCHY
Enter Plain Text: HELLO
Cipher Text: CFSUPM

```